Once Rolling Hashing is Enough: Exploiting Rolling Hash Reuse in Delta Compression

Haoliang Tan
Harbin Institute of Technology
Pengcheng Laboratory
Shenzhen, China

Wenhao Ou
Harbin Institute of Technology
Shenzhen, China

Xiangyu Zou
Harbin Institute of Technology
Shenzhen, China

Cai Deng
Harbin Institute of Technology
Shenzhen, China

Yanqi Pan
Harbin Institute of Technology
Shenzhen, China

Hao Huang
Harbin Institute of Technology
Shenzhen, China

Zhaoquan Gu
Harbin Institute of Technology
Pengcheng Laboratory
Shenzhen, China

Wen Xia*
Harbin Institute of Technology
Pengcheng Laboratory
Shenzhen, China

Abstract

In backup storage, delta compression successfully achieves a much higher data reduction ratio than chunk-level deduplication by applying a finer granularity in redundancy detection and elimination. However, it introduces additional, intensive computation overhead and results in a 50%–70% worsening backup throughput.

We find that the computational inefficiency is due to the redundant byte-wise rolling hash computation across multiple stages of the delta compression, although they are mergeable. We therefore present FastDelta, a computationally efficient framework for delta compression that proposes a novel *sampling-based rolling hash reuse* approach. By caching and reusing sampled rolling hashes to avoid redundant computation, FastDelta achieves high-performance delta compression with a low memory overhead. Moreover, we propose a series of techniques, such as embedding-style sampling operation and locality-aware container compression, which mitigate the performance overhead and data reduction loss caused by sampling-based hash reuse. Evaluations show that FastDelta achieves 1.3×–2.1× higher end-to-end throughput while preserving a comparable data reduction ratio.

CCS Concepts: • **Information systems** → **Deduplication**; • **Software and its engineering** → *Delta compression.*

*Corresponding author. Email: xiawen@hit.edu.cn.

EUROSYS '26, Edinburgh, Scotland Uk

ACM ISBN 979-8-4007-2212-7/2026/04
https://doi.org/10.1145/3767295.3803596

ACM Reference Format:
Haoliang Tan, Wenhao Ou, Xiangyu Zou, Cai Deng, Yanqi Pan, Hao Huang, Zhaoquan Gu, and Wen Xia. 2026. Once Rolling Hashing is Enough: Exploiting Rolling Hash Reuse in Delta Compression. In *21st European Conference on Computer Systems (EUROSYS '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3767295.3803596

1 Introduction

Backup storage plays a critical role in enterprise data protection [12, 30, 50]. However, the explosive growth of backup data [32] has made storage costs a major concern [10, 40, 41].

Chunk-level deduplication [14, 17, 20, 23] has been widely used in enterprise backup storage [10, 19, 28] to reduce the backup data volume. Specifically, it splits files into chunks (e.g., 8KB average size) by content-defined chunking (CDC) [29, 46] and identifies and removes duplicated chunks by their secure fingerprint (e.g., SHA-1 [13]).

However, its compression effectiveness is constrained by the coarse chunk granularity. To further eliminate sub-chunk redundancies in similar but non-identical chunks, delta compression [11, 33, 48, 52] has been introduced on top of chunk-level deduplication, delivering an additional 2×–4× compression gain [34, 42, 48].

Typically, delta compression employs "*similarity detection*" to extract non-identical chunks' similarity features to find their similar base chunks. After that, it runs "*delta encoding*" to compute the fine-grained difference between the input chunk and its similar base chunk, generating a much smaller delta chunk. However, these two extra steps introduce substantial computational overhead [43, 49, 51] and greatly slow down the backup throughput, as shown in Figure 1.

Many solutions (e.g., Finesse [49], Odess [51], Xdelta [24], and Gdelta [36]) have been proposed to design more lightweight algorithms on similarity detection or delta encoding,

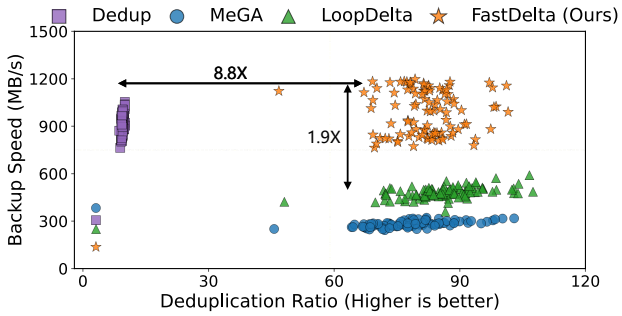


Figure 1. Performance of chunk-level deduplication (Dedup), SOTA delta compressions (MeGA [52], LoopDelta [48]), and our approach FastDelta on Chromium project backups. *FastDelta achieves high deduplication ratios of delta compression with fast backup speed close to chunk-level deduplication. Evaluations are conducted on a workstation running Ubuntu 18.04, equipped with an Intel Xeon Silver 4215R CPU (detailed in §5.1). All methods are configured to use 4 cores by default.*

but they still result in considerable computation cost. Specifically, on four widely-studied backup datasets, similarity detection and delta encoding account for 44%–47% (see §3.1) of the total computation time in an advanced delta compression framework (i.e, MeGA [52]), which is 1.0×–1.8× higher than that of chunk-level deduplication, i.e., chunking and fingerprinting stages.

To identify the root cause of the inefficient computation overhead, we make a deep analysis of similar detection and encoding. We find that their multiple rolling hash computation dominates the overhead. Typically, rolling hash operations account for approximately 50%–90% of the total cost in similarity detection, and 40%–50% in delta encoding across state-of-the-art techniques [24, 36, 46, 49, 51] (see §3.1).

After revisiting the delta compression workflow, we identify an opportunity to eliminate this inefficiency. Typically, **three stages in delta compression (i.e., content-defined chunking, similarity detection, and delta encoding) repeatedly perform the rolling hash computation on the backup stream.**

These stages all require fine-grained content characterization, which is usually achieved by running rolling hash functions byte-by-byte in existing approaches [42, 48, 52]. However, since the stages invoke rolling hash computations independently, they incur redundant overhead.

To address this inefficiency in delta compression, we propose **FastDelta**, with the key idea of *sampling-based rolling hash reuse*. It unifies the rolling hash computation across three critical stages in delta compression by computing rolling hash once during CDC stage, recording its hash results based on sparse content-based sampling, and reusing them in subsequent similarity detection and delta encoding stages.

The efficiency of *rolling hash reuse* with *sparse content-based sampling* in FastDelta comes from several aspects.

(1) FastDelta makes a space-for-computation tradeoff by reusing CDC rolling hash results to entirely eliminate redundant byte-wise rolling hash computations in the extra stages of delta compression, which leads to multi-fold acceleration in similarity detection and delta encoding.

(2) FastDelta applies sparse sampling to reduce the reuse hash volume to an acceptable level. This avoids directly reusing all rolling hash results, which can overwhelm the system resources, i.e., excessive memory for recording immediately reused hash results and external storage for storing hashes at an unpredictable reuse time. The reason is that rolling hash processes data byte-by-byte and produces a hash value (e.g., 8 bytes) on each byte of data. These hash results are several times larger than the backup image itself (usually tens of gigabytes or more [19]), which makes the reuse of full rolling hashes impractical.

(3) FastDelta employs content-based sampling to achieve comparable similarity detection accuracy and incur only a slight compression loss in delta encoding when these stages reuse only sparse rolling hash values.

To alleviate the negative effect of sampling-based hash reuse, several techniques are proposed in FastDelta: (1) An embedded sampling operation to maintain high CDC speed under hash sampling; (2) A locality-aware container compression technique to compensate for the compression loss caused by hash sampling in delta encoding; (3) A lifecycle-based hash retention mechanism with piggybacked I/O to resolve hash metadata explosion and reduce hash retrieval overhead from external storage.

As shown in Figure 1, FastDelta achieves performance close to chunk-level deduplication while preserving the appreciated deduplication ratio benefit. The contributions of this paper are fourfold.

- We reveal a fundamental inefficiency in delta compression: the redundant and computationally intensive rolling hash operations across its three critical stages.
- We eliminate the rolling hash computation of similarity detection and delta encoding by reusing CDC's hash results based on sparse content-based hash sampling.
- We propose three techniques of embedded hash sampling operation, locality-aware container compression, and piggybacking I/O with lifecycle-based retention for hash metadata to complete the sampling-based hash reuse.
- We propose FastDelta with these techniques, accelerating similarity detection and delta encoding by 10×–100× and 3×–9×, respectively, enhancing the system performance by 1.3×–2.1× while preserving a high deduplication ratio.

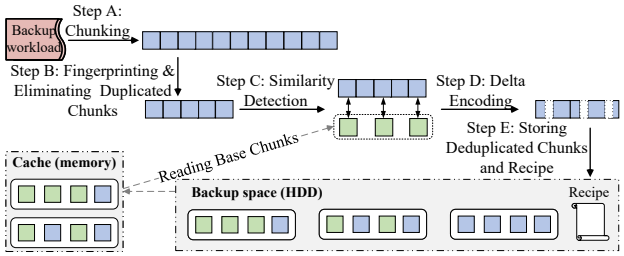


Figure 2. The backup workflow of delta compression. *This work targets at eliminating redundant rolling hash computation at the stages of similarity detection and delta encoding.*

2 Background and Related Works

2.1 Delta Compression

Delta compression [15, 37, 48, 52] could achieve a much higher deduplication ratio than chunk-level deduplication alone in backup storage. It focuses on sub-chunk-level redundancy detection and elimination, based on the chunk-level deduplication. However, this capability comes at the cost of considerable computational and I/O overhead.

General workflow. Figure 2 shows a standard delta compression workflow for processing incoming workloads on the backup storage side: ❶ **Chunking.** Split the backup stream into chunks by content-defined chunking. ❷ **Deduplication.** Calculate a secure fingerprint for each chunk. Check and eliminate duplicated chunks using the fingerprint index. ❸ **Similarity detection.** Calculate similarity features (combined as sketches) for unique chunks. Use the sketch index or cache to find similar candidates for unique chunks. ❹ **Delta encoding.** If a similar candidate is found, treat it as a base chunk and delta encode the incoming chunk, often resulting in a much smaller delta chunk. ❺ **Writing.** Store all deduplicated chunks in containers (i.e, the basic storage unit of deduplicated chunks, usually with 4MB size [53]), which are then compressed. Generate a backup recipe by recording all chunks' metadata for restoring.

2.2 Computation-oriented Optimizations

Rolling hash. A rolling hash function is designed to efficiently update the hash value of a byte-wise sliding window over data instead of recomputing the hash from scratch. Rabin hash[31] is a classic representative, which was further popularized by the well-known Karp-Rabin string matching algorithm [18]. Rabin is also widely employed in CDC [27, 48], similarity detection [8, 49], etc, but with low speed. Gear [43] proposes a much simpler method for rolling hash updates, formulated as, `hash = (hash+Table[new_char]) ≪ 1`, with `Table` being a precomputed random values mapping table. This significantly improves the rolling hash speed and provides a comparable random hash distribution with Rabin hash at the same time.

Content-defined chunking. CDC is proposed to resolve the "boundary shift" problem [27] of fixed-size chunking (FSC). Typically, CDC declares chunk boundaries based on byte contents instead of on byte offset using rolling hash. CDC detects about 10%–20% more redundancy than the FSC approach. FastCDC [46] employs Gear rolling hash to characterize the chunk's content and declares the chunking point if the byte-wise hash value satisfies a predefined condition, e.g., `hash & pre_defined_mask == 0`. FastCDC has been employed in many practical projects, e.g., Ceph [1], Google File Transfer [2], ZSTD [3], and deduplication studies [15, 19, 35, 47], due to its fast speed. Thus, our work also chooses Gear-based FastCDC as the basis chunking method for rolling hash reuse.

Similarity detection. N-transform [8] is a classic chunk-level similarity detection method. It computes byte-wise Rabin hashes on the chunk (similar to CDC), and then each of them is linearly transformed N times to calculate N hash value sets. Finally, N maximal values of each set are selected as features.

Finesse [49] eliminates the linear transform cost by dividing the chunk into fix-sized sub-chunks and selecting the maximal value of each sub-region as a feature. However, this sacrifices some similarity accuracy because it selects features from a randomly sampled subset rather than from the global set of hashes, thereby violating Broder's theorem [7]. The theorem states that the probability that two sets share the same minimum hash value equals the similarity between the two sets.

Odess [51], a SOTA speed-optimized variant, replaces the heavy Rabin hash with the lightweight Gear hash and introduces a content-based sampling scheme that transforms only the selected hashes, greatly reducing the number of transformations while preserving comparable similarity detection accuracy. To expedite feature comparisons, features are usually combined into fewer "Super Features" as sketches to efficiently identify highly similar chunks, as suggested in [42, 49, 51].

Delta encoding. Xdelta [24] is widely used in delta compression [15, 42, 48, 52] as a chunk-level delta encoder. Xdelta identifies duplicated strings (also called words) between similar chunks and encodes them using compact instructions for compression. This begins by scanning the base chunk to generate overlapping words, each representing the string content of a sliding window, with Adler32 rolling hash [25], and builds a hash table that maps word hash values to word positions, serving as a "base chunk dictionary". Word hashes of the input chunk are also generated with Adler32 and used to probe the dictionary to retrieve the corresponding word positions in the base chunk, enabling word content comparison to identify duplicated words between two similar chunks. Consequently, delta encoding requires both the

rolling hash values and positions for duplicated word matching. Gdelta[36] and Edelta[44] improve performance by replacing Adler32 with the more efficient Gear hash. Additionally, these methods often apply length-based sampling to reduce the number of word hashes indexed or word matching for acceleration.

2.3 I/O-oriented Optimizations

Compared to chunk-level deduplication, delta compression significantly reduces IO write bandwidth required for deduplicated data in the backup process due to a higher deduplication ratio, but incurs reading base chunks I/O for delta encoding and decoding in the backup and restore processes, respectively.

To alleviate restore I/O overhead, MeGA [52] proposes a lifecycle-based data layout, which groups chunks based on their consistent reference patterns across backup workloads. This classification ensures that chunks in the same category are either always needed together or not at all during restoration. To reduce backup I/O overhead, MeGA [52] proposes filtering those "sparse containers" with fewer base chunks, to reduce base chunks I/O. LoopDelta [48] optimizes the backup I/O cost to be close to chunk-level deduplication through a piggybacking I/O technique, which loads base chunks by piggybacking them on the metadata routine I/O during deduplication.

These advanced delta compression frameworks, focusing on I/O optimization, have successfully shifted the performance bottleneck of delta compression to computation, as studied in §3.1. Notably, FastDelta can be easily integrated into these I/O-optimized delta compression frameworks (e.g., MeGA, LoopDelta) to enhance their computational efficiency.

2.4 Backup Storage

Backup storage [5, 6, 39] is typically implemented as a standalone storage server, either as a purpose-built backup appliance or as a virtual machine instance, optimized for reliable data protection and efficient backup operations [10].

Backup version. In backup storage [26], workloads typically consist of a series of backups (i.e., successive snapshots of primary data), and consecutive backups tend to be highly similar, as reported in [42, 48, 49, 52]. This high redundancy often leads backup storage systems to employ data deduplication techniques, greatly reducing backup sizes and saving hardware costs. To optimize metadata management, garbage collection, and indexing, some works [21, 52, 53] only perform deduplication against the previous backup version.

Container. Deduplicated data (i.e., chunks) is usually compressed and stored in immutable and fixed-size containers (e.g., 4MB). Containers are compatible with striping across multiple drives in a RAID configuration and writing in large units to maximize sequential throughput [22]. It is the basic unit of chunks being read from and written to the storage.

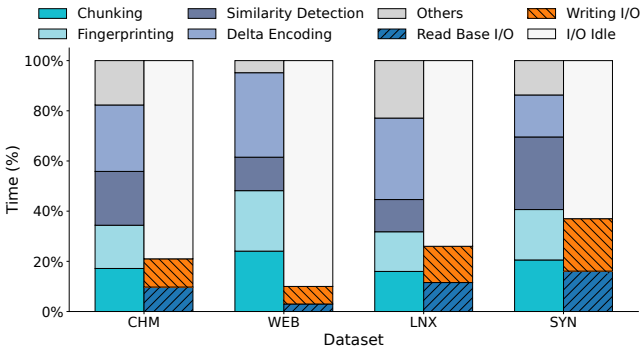


Figure 3. A time cost breakdown of delta compression. *"Others" presents the computation of fingerprint, sketch indexing, and cache updating, etc. "Read base" and "Writing" denote the I/O time (%) of reading the container for base chunks and writing deduplicated data compared to total computation. Left bar: computation, Right bar: disk I/O.*

Content-defined chunking			
Approach	Rolling hash	Speed (MB/s)	Roll-hash COMP. (%)
FastCDC [46]	Gear [43]	1800	95.2
Similarity Detection			
Approach	Rolling hash	Speed (MB/s)	Roll-hash COMP. (%)
N-transform [8]	Rabin [31]	100	57.9
Finesse [49]	Rabin	400	85.5
Odess [51]	Gear	1000	90.8
Delta encoding			
Approach	Rolling hash	Speed (MB/s)	Roll-hash COMP. (%)
Xdelta [24]	Adler32 [25]	300	47.4
Edelta [44]	Gear	400	45.1
Gdelta [36]	Gear	700	39.6

Table 1. The rolling hash configuration, average speed, and rolling hash computation percentage of existing SOTA CDC chunking, similarity detection, and delta encoding methods. *MeGA [52] is configured with FastCDC, Odess, and Xdelta.*

3 Observations and Motivations

3.1 Computation is the Performance Bottleneck in Delta Compression

As discussed in §2, delta compression obtains a higher deduplication ratio than chunk-level deduplication. However, it incurs extra computation of similarity detection and delta encoding, and read base chunk I/O overhead, which greatly worsens its backup throughput.

To identify these extra overheads, we run the latest open-source delta compression framework, MeGA [52], on four studied backup datasets (detailed in Table 2) and collect the breakdown of computation and disk I/O time. Figure 3 shows the time cost breakdown. It reveals two points.

- **Backup performance is CPU-bound.** The total computational cost exceeds the I/O cost (i.e., reading base chunks and writing deduplicated chunks) by a factor of approximately 2.7× to 10×. The reason is that delta compression

is applied to the entire backup stream, resulting in much smaller, deduplicated data. This significantly reduces the write I/O overhead. This phenomenon is more apparent in backups with high deduplication ratios, such as the WEB dataset (100× as evaluated in §5.3). On the other hand, read I/O of base chunks can be mitigated via a memory-resident container cache, while the extra computation of delta compression offers no shortcut. Meanwhile, the advanced delta compression frameworks further reduce the read base chunk I/O by disabling delta compression on the sparse base chunk containers in MeGA [52] and piggybacking base chunk I/O in LoopDelta [48].

- **Delta compression greatly increases the computation cost.** Chunking and fingerprinting serve as the primary bottlenecks in chunk-level deduplication [46], while delta compression additionally introduces non-negligible computation cost, increasing the overhead by 1.0×–1.8×. Although similarity detection (i.e., Odess [51]) is faster than delta encoding (i.e., Xdelta [24]), as shown in Table 1, it involves processing a larger number of data chunks. The former processes all non-identical chunks, while the latter processes only similar chunks. Thus, their computational overhead is comparable, and both require acceleration.

These findings motivate us to reduce the additional computation costs of delta compression to improve the backup efficiency. Especially in the context of cost-effective backup storage, reducing computation not only improves system throughput but also frees up CPU resources.

3.2 Key Observation: Computation Inefficiency in the Current Delta Compression Framework

As discussed in §2.2, many algorithms [36, 44, 49, 51] have been proposed to reduce the computation cost of delta compression. However, they focus solely on optimization in single stages, i.e., similarity detection and delta encoding, which yields diminishing returns and still results in considerable computation overhead, as analyzed in §3.1.

Fortunately, by revisiting the entire delta compression workflow, we identify a fundamental computational inefficiency in the workflow, which can be exploited to significantly reduce computation. We observe that three critical stages in delta compression, i.e., CDC chunking, similarity detection, and delta encoding, repeatedly apply the heavy byte-wise rolling hash computation, to characterize the chunk's content. More specifically, each byte-wise rolling hash represents the content of a sub-chunk (sliding window). As shown in Figure 4, the workflow of delta compression is:

(1) In CDC chunking, a rolling hash is applied over the backup stream, and it declares chunking boundaries if hash values satisfy the pre-defined chunking condition, which is seen as selecting a sub-chunk as a chunking point.

(2) In similarity detection, a rolling hash is first run on the non-identical chunk, and the maximum value of the rolling

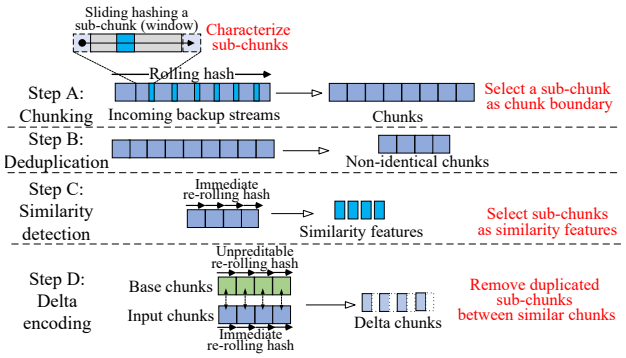


Figure 4. Repeated rolling hash computation in delta compression. *It is classified into two types: immediate re-execute after CDC and re-execute after CDC at an unpredictable time.*

hash set is chosen as a feature, which is seen as selecting a sub-chunk as a feature, for matching similar chunks.

(3) In delta encoding, it first executes a rolling hash over the base chunk. The content within each sliding window is referred to as a "word", i.e., a sub-chunk. The base chunk's word positions are indexed by word hashes in a hash table. Then, the same rolling hash is applied to the input chunk again to generate its word hashes, used to probe the hash table to match duplicated words between similar chunks, i.e., locate and remove their duplicated sub-chunks (words).

Therefore, in the existing delta compression workflow, three stages of CDC, similarity detection, and delta encoding repeatedly run a byte-wise rolling hash. Although they serve different purposes, they all aim at characterizing (sub-) chunk content with the hashes. This makes the repeated computations redundant. Table 1 further shows that rolling hash computation accounts for 50%–90% in similarity detection and 40%–50% in delta encoding, across existing techniques. This indicates that rolling hash computation is a major overhead in the two extra stages of delta compression. These findings motivate us to avoid redundantly computing the expensive rolling hash for the same objective, i.e., characterizing chunk content, across different stages in delta compression.

Notably, prior work (e.g., [45]) attempts to employ SIMD for parallel rolling hash computation. However, due to the byte-level computation dependency in rolling hash, its speedup is much lower than FastDelta, as evaluated in §5.2.2.

3.3 Motivation: Sparse Content-based Hash Sampling for Rolling Hash Reuse

To avoid redundant rolling hash computation, we propose reusing rolling hash results across three stages in the delta compression framework. Specifically, it is achieved by only computing rolling hash once in CDC chunking, and eliminating the rolling hash computation of similarity detection and delta encoding by reusing CDC's hash results.

To enable hash reuse, Both memory (DRAM) and external storage (HDD) are required. Specifically, as shown in Figure

4, memory is required for short-term hash reuse, i.e., similarity detection and delta encoding on input chunks. These processes' rolling hashes are immediately re-executed after CDC. Thus, we can cache the input chunk's CDC rolling hash results for immediate reuse. In contrast, the rolling hash over the base chunk in delta encoding is re-executed at an unpredictable time after its CDC process. Thus, external storage is required for storing their rolling hashes to support long-term reuse.

Challenge: Massive hash volume makes hash reuse impractical. However, directly reusing rolling hash results can overwhelm the system resources (i.e., memory and storage) due to the massive hash volume. Typically, rolling hash generates byte-wise hashes over data. Thus, recording byte-wise 32-bit hash position (for word matching in delta encoding) and 64-bit hash value of rolling hash over a backup stream leads to a 12× more volume of hash data than the backup stream itself. As a result, short-term hash reuse requires up to 12× more memory than backup streams, while long-term hash reuse consumes even more space than backup data, making hash reuse impractical in delta compression systems.

Sampling-based hash reuse. To address the massive hash volume problem, we conduct a deeper analysis of delta compression techniques. We find that the subset of byte-wise rolling hash values is adequate to achieve comparable similarity detection accuracy and incurs only a slight compression loss in delta encoding. For example, in existing similarity detection and delta encoding methods, byte-wise rolling hashes are typically computed first, but sampling is often applied to reduce the number of hashes used in later computations.

This indicates that rolling hash reuse can be achieved under two conditions: (1) **Sparse**. Intermediately reused hashes are sampled very sparsely. For example, a 1/256 sampling rate can reduce the original 12× memory overhead to just 5%. (2) **Effective**. Similarity detection and delta encoding remain effective with sparse hashes. By analyzing the sampling mechanism in existing methods, we propose our hash reuse scheme at the end of this subsection.

In existing methods, sampling can be classified into length-based sampling and content-based sampling.

Sampling in similarity detection. As discussed in §2.2, N-transform is a classic method without sampling. It applies N linear transforms to all byte-wise rolling hashes, to generate N features. In contrast, Odess [51] also generates byte-wise rolling hashes, but only applies linear transforms on the selected hashes, sampled by hash values, to reduce computation. It maintains comparable detection accuracy with N-transform even under sparse sampling, as studied in [51]. This prior work suggests the feasibility of rolling hash reuse in similarity detection with content-based sampling, as it perfectly satisfies both sparse and effective.

Sampling in delta encoding. Existing delta encoding, e.g., Xdelta [24] and Gdelta [36], typically use a length-based

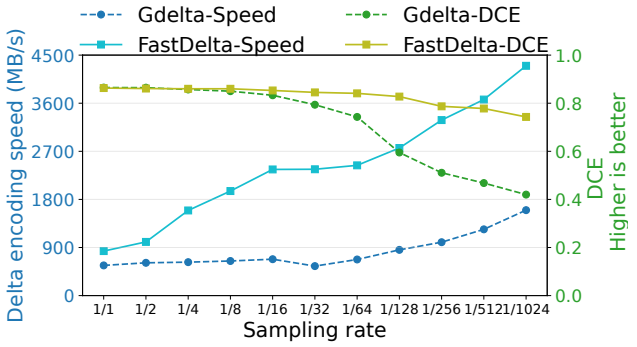


Figure 5. Delta encoding performance. *The length-based sampling (Gdleta) and content-based sampling (FastDelta) methods are evaluated on the web snapshot dataset.*

sampling scheme to reduce computation by sacrificing some compressibility. For example, Xdelta first generates byte-wise word hashes for base chunks, but it selectively indexes these hashes, based on a fixed word interval, into the hash table. Gdelta similarly computes byte-wise word hashes for input chunks, but performs word matching only at fixed steps. This length-based sampling scheme is common in compression algorithms to build a very simple trade-off between finding more redundancy and reducing computation, by controlling a parameter of the fixed interval.

Without considering hash reuse, this scheme is efficient because it significantly reduces the computation cost of byte-wise word indexing or word matching, while still retaining a sufficient number of words for accurate matching between similar chunks, at small step sizes.

However, this scheme is not suitable for our hash reuse, which samples words in both input chunk and base chunk simultaneously, with much sparser sampling. The problem is that duplicated words in similar chunks are often located at different offsets due to data modifications, leading to misalignment. Hence, word matching suffers from the "boundary shift" problem at length-based sampling. It cannot capture duplicated words with different offsets in similar chunks at the same time, resulting in severe compression loss.

Specifically, Figure 5 presents the delta compression efficiency (DCE) of the length-based sampling method (Gdelta) under different sampling rates. DCE represents the proportion of redundancy eliminated of input chunks by delta encoding, calculated by $(1 - \frac{\texttt{chunk\_size\_after\_delta\_encode}}{\texttt{chunk\_size\_before\_delta\_encode}})$, $\in [0, 1]$. Results show that the DCE of Gdelta decreases marginally at large sampling rates, but drops sharply at small sampling rates, with a reduction of up to 42%. This degradation occurs because sparse sampling reduces the chance of simultaneously selecting non-aligned words from similar chunks, exacerbating the "boundary shift" problem.

Inspired by the CDC, which declares the chunk boundary based on its content to resolve the "boundary shift" problem in FSC, and also Odess, which maintains comparable similarity accuracy with content-based hash linear transforms, we

propose a content-based sampling word-matching scheme for delta encoding. Specifically, it samples words based on its content, which is achieved by a word hash sampling condition judgment (i.e., `word_hash & mask == 0`), enabling consistent sampling of identical words even when their positions are misaligned. As shown in Figure 5, our scheme improves the DCE by 50%–60% under sparse sampling rates. Also, by eliminating rolling hash computations, the delta encoding speed is greatly boosted, as expected.

To this end, under ***content-based hash sampling***, both similarity detection and delta encoding meet the two key requirements for reusing hash results from CDC, i.e., *adequate effectiveness* with *sparse* hash values. This ensures that hash reuse does not overburden the system resources.

Remaining challenges. However, sampling-based hash reuse still introduces three issues that need to be addressed. (1) Content-based sampling introduces an additional byte-wise sampling condition judgment in CDC, resulting in CDC speed degradation. (2) Hash sampling inevitably misses some matching of duplicated words during delta encoding, resulting in DCE degradation. (3) Long-term hash reuse requires external storage and also extra retrieval I/O when accessing these hashes. We propose several techniques to address these issues, which are introduced in the next section.

4 Design and Implementation

4.1 FastDelta Overview

Figure 6 shows the overall FastDelta backup framework. The workflow is as follows: ❶ **Chunking**. FastDelta first applies *embedded-style sampling* in CDC (§4.2) to split the stream into chunks and record the *content-based sampled* rolling hash values for reuse in subsequent steps. ❷ **Deduplication**. Duplicate chunks are eliminated based on the secure fingerprint. ❸ **Similarity Detection.** Similarity detection is performed on non-duplicated chunks with *rolling hash reuse* (§4.3) to generate similarity features for identifying similar matches. ❹ **Delta Encoding.** Delta encoding is run to compute differences between similar chunks with *rolling hash reuse* (§4.4). The *inter-chunk delta-encoded* chunks and unique chunks are placed into containers for further *locality-aware compression*. ❺ **Writing**. Chunks and *lifecycle-based* rolling hashes are stored in containers for *piggybacking I/O* (§4.5), with all data subsequently written to disk.

4.2 Embedded Hash Sampling in CDC

FastDelta adopts FastCDC [46] to partition data into chunks, while simultaneously recording the rolling hash values and positions for reuse. This design removes redundant rolling hash computations and thus speeds up similarity detection and delta encoding.

As discussed in §3.3, directly recording all hash values and positions leads to prohibitive memory overhead, which is impractical for delta compression systems. To mitigate

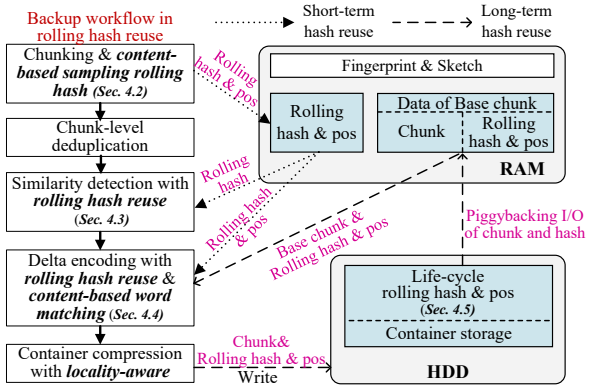


Figure 6. FastDelta overview with hash reuse. *Short-term hash reuse is for eliminating re-executed rolling hash immediately after CDC. Long-term hash reuse is for eliminating re-executed rolling hash at an unpredictable time after CDC.*

this, hash values must be sampled at a low rate to reduce memory consumption. To align with our content-based sampling mechanism of similarity detection (see §4.3) and word-matching in delta encoding (see §4.4), we incorporate content-based sampling into CDC to record selected hash values.

However, introducing content-based sampling adds an extra byte-wise condition check in CDC, in addition to the original chunking boundary judgment. This effectively doubles the number of condition evaluations per rolling hash, resulting in degraded performance due to frequent branch mispredictions and disrupted instruction-level parallelism on CPUs. As shown in Figure 7, the added sampling check reduces FastCDC throughput by more than 50%. We further experimented with replacing the sampling condition branch with conditional move instructions (CMOV) [4], but observed no improvement. The reason is that both sampling implementation variants ultimately result in the same branch-based assembly code, and the branch prediction overhead remains identical. Consequently, the CMOV-style implementation exhibits no performance difference compared to the original conditional branch version.

This bottleneck undermines our goal of eliminating repeated rolling hash computations, as CDC continues to dominate computation in deduplication-based systems (§3.1). To resolve this, we propose embedding the sampling judgment directly into the chunking condition.

As shown in Algorithm 1, FastCDC [46] determines chunk boundaries using a mask set to 0X7FF3 (as an example). A boundary is declared once the condition (`hash & 0X7FF3 == 0`) is satisfied (line 10), indicating that the sliding window aligns with a valid boundary. This 13-bit mask corresponds to an average chunk size of 2^{13} bytes, i.e., $L = 8$, KB.

To support sampling, we introduce an additional 8-bit mask (0X0FF0), which yields a sampling ratio of $2^8 = 256$, denoted as S. The sampling condition (`hash & 0X0FF0 == 0`)

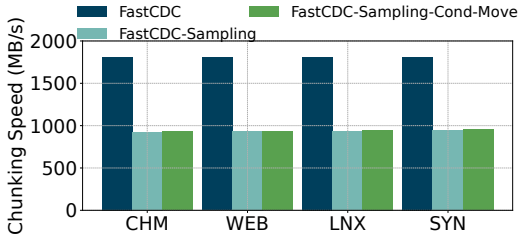


Figure 7. Average chunking speed of FastCDC with/without byte-wise rolling hash sampling operations. *"Sampling" denotes directly adding the sampling condition judgment. "Cond-Move" denotes a branch-free hash sampling version.*

Algorithm 1: Embedding hash sampling in content-defined chunking of FastDelta

Input: Chunk content Str, length of the chunk, L, a predefined table for computing Gear hash, Gear[]
Output: Chunking Pos, Hash[], Hash Position[], Hash num

```
1  ChunkingMask ← 0X7FF3 ;              // Chunking mask
2  EmbedRecordMask ← 0X0FF0 ;    // Embedded mask of
     chunking mask for content-based sampling hash
3  FP ← 0;
4  for i = 0; i < L; i ++ do
5      FP ← (FP ≪ 1) + Gear[STR[i]] ;   // Rolling hash
6      if FP & EmbedRecordMask == 0 then
7          Hash[cnt] ← FP ;     // Sampling hash values
8          Pos[cnt] ← i ;     // Sampling hash positions
9          cnt ← cnt + 1;
10         if FP & ChunkingMask == 0 then
11             return i, Hash[], Pos[], cnt ;    // Chunking
```

is embedded directly into the chunking judgment. Because the sampling mask is a strict subset of the chunking mask, any position that satisfies the chunking condition necessarily satisfies the sampling condition as well. Consequently, embedding the sampling judgment preserves both the original chunking boundaries and the overall deduplication ratio.

By embedding the sampling checks into the chunking condition, the system ensures that chunking is evaluated only when the sampling condition holds. This reduces the number of additional checks to merely L/S, thereby preserving FastCDC's high throughput, as demonstrated in §5.2.1.

Furthermore, this integrated sampling mechanism extracts L/S content-based sampled rolling hash values and chunk positions for reuse in subsequent similarity detection and delta encoding. For each backup stream, a contiguous in-memory array is allocated to cache the sampled hashes, with a size of $\frac{\texttt{Data\_Size}\times(8\text{Bhash}+4\text{B}_{\text{pos}})}{\texttt{Sampling\_Ratio}}$, which corresponds to an additional ~5% memory overhead at a sampling ratio of 256. Each chunk records metadata that specifies the offset and number of its associated hashes in this array. The memory space for both the array and the backup stream is allocated and reclaimed in tandem.

Algorithm 2: Similarity Detection of FastDelta

Input: Rolling hash array Hash[], hash num cnt, random value pairs for linear transformation (m_i, a_i)
Output: N similarity features Features[N]

```
1  Features[] ← empty;
2  FP ← 0;
3  for i = 0; i < cnt; i ++ do
4      FP ← Hash[i] ;              // Reuse rolling hash
5      for j = 0; j < N; j ++ do
6          Transform[j] ← (m_i × FP + a_i) mod 2^32 ;
             // Linear transformations for N times
             only on sampled hashes
7          if Transform[j] > Features[j] then
8              Features[j] ← Transform[j];
9  returen Features[N];
```

4.3 Similarity Detection with Rolling Hash Reuse

Similarity detection is designed to extract similarity features from non-duplicated chunks in order to identify potential matches. As illustrated in Algorithm 2, FastDelta differs from traditional approaches (e.g., N-transform [8] and Odess [51]) by avoiding repeated rolling hash computations (e.g., Gear [43] and Rabin [31]). Instead, it directly reuses the precomputed hash values generated during the CDC process, thereby substantially reducing computational overhead. Consequently, similarity detection in FastDelta only involves lightweight hash linear transformations.

Furthermore, FastDelta inherits the speed advantage of Odess by requiring fewer linear transformations. Instead of applying transformations to all rolling hashes, it operates only on the subset sampled in CDC. Concretely, the number of linear transformations in FastDelta is $\frac{\text{Chunk\_Size}\times\text{Features\_Num}}{\text{Sampling\_Ratio}}$, which produces N similarity features. This design resembles Odess and achieves a detection capability comparable to the classical N-transform [51].

In addition, FastDelta improves upon Odess by eliminating the conditional judgment for hash sampling. Since hash sampling is already integrated into the CDC process, the similarity detection stage avoids extra conditional checks, further boosting computational efficiency. Finally, similarity features are selected as the maximum values within their respective linear transformations.

4.4 Delta Encoding with Rolling Hash Reuse

Delta encoding aims to capture and store only the differences between similar chunks, thereby avoiding the repeated storage of base chunks. This subsection first reviews the workflow of conventional delta encoding approaches (e.g., Xdelta [24]) to establish the baseline, and then introduces

Algorithm 3: Delta Encoding of FastDelta

Input: Base chunk bas[], Input chunk ipt[], Rolling hash of base and input chunk (Bhash[], Ihash[]), Rolling hash position (Bpos[], Ipos[])
Output: Delta chunk, dlt[]

```
1  anchor ← 0;
2  hashTable[] ← empty;
3  for i = 0; i < size(Bhash[]); i++; do
4      hashTable[Bhash[i] & indexMask] ← Bpos[i] ;
         // Build word hash index of base chunk
5  for j = 0; j < size(Ihash[]); j++; do
6      index ← Ihash[j] & indexMask;
7      offset ← hashTable[index] ;          // Get word
         position of base chunk
8      for mLen ← 0; ; mLen ++; do
9          if bas[offset+mLen]!= ipt[Ipos[j] +mLen] then
10             break ;        // Word content comparison
11     if mLen ≥ wordSize then
12         insertIns ← j − anchor ;        // INSERT INST
             (unmatched literals length)
13         dlt ← dlt + insertInst;
14         memcpy(dlt+dlt.Len, ipt+anchor,instIns.Len) ;
             // Save the unmatched literals
15         copyIns ← (mLen, offset) ;        // COPY INST
16         dlt ← dlt + copyInst;
17         anchor ← Ipos[j] + mLen;
18 return dlt;
```

the FastDelta workflow, highlighting how it streamlines and accelerates the encoding process.

Conventional delta encoding typically consists of three steps: (1) Characterize: A rolling hash is used to generate words (i.e., sliding-window contents) from base chunks, and each word's position is indexed into a hash table. Words of the input chunk are generated in the same way. (2) Matching: Each word hash of the input chunk is queried against the base chunk's hash table, followed by content comparison at the corresponding positions to identify matches. (3) Encoding: Matched words are encoded as copy instructions, while unmatched words are encoded as insert instructions and stored in the delta. For further compression, Xdelta applies Huffman coding to the unmatched words.

Algorithm 3 outlines the overall workflow of FastDelta, which is elaborated below.

In the *Characterize* step, FastDelta eliminates rolling hash computation for both input and base chunks. Specifically, hashes of the input chunks are directly reused from the CDC process, while base chunk hashes are retrieved from external storage via piggybacking I/O alongside base chunks. A hash table is then constructed (lines 4–5), mapping each rolling hash to its position in the base chunk for word matching.

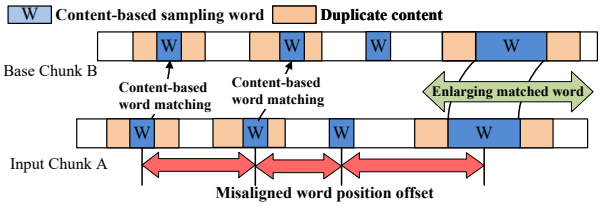


Figure 8. An example of word-matching used content-based sampling in FastDelta. *It captures misaligned duplicated words between similar chunks based on content instead of position.*

Thanks to FastDelta's sparse hash sampling, building this sampled index is much faster than indexing all rolling hashes.

In the *Matching* step, unlike existing methods [24, 36] that query the base chunk's hash table at byte-level or small fixed-length intervals for content comparison, FastDelta adopts a sparse content-based sampling strategy for word matching. By sampling sparsely, FastDelta performs far fewer word matches than byte-wise or small-length (several bytes) methods, thereby greatly reducing both word indexing (lines 6–8) and content comparison (lines 9–11) overhead.

As illustrated in Figure 8, content-based sampling is also more effective in mitigating the boundary shift problem, where identical words in similar chunks may appear at misaligned positions. Length-based sampling methods often fail to capture these words under sparse sampling rates due to misalignment. In contrast, content-based sampling selects words according to content rather than fixed positions, making it more likely to detect such shifted matches. When applied symmetrically to both input and base chunks, this scheme only slightly reduces the compressibility of delta encoding, as analyzed in §3.3.

In the Encoding step, FastDelta follows the conventional design of using copy and insert instructions: duplicated words between the input and base chunks are encoded as copy operations (i.e., references to duplicate content), while unmatched literals are stored in the delta chunk.

To alleviate the compression loss caused by hash sampling, we introduce a technique called locality-aware container compression, which exploits the interplay between delta encoding and container-level compression. We observe that both stages compress delta chunks independently: (1) Conventional delta encoding (e.g., Xdelta) applies entropy coding such as Huffman [16] to compress unmatched words within each delta chunk. (2) After delta chunks are packed into containers, a general-purpose compressor (e.g., ZSTD [9]) is applied again at the container level. However, applying entropy coding too early can diminish the effectiveness of container-level compression. Container compression benefits from exploiting redundancy across adjacent chunks due to locality, but this opportunity is largely lost once delta

encoding transforms the chunk into entropy-coded, pseudo-random content. To preserve locality, FastDelta disables intra-chunk entropy coding during delta encoding and instead relies on container-level compression to exploit cross-chunk redundancies. This design significantly narrows the delta compression efficiency (DCE) gap at sparse sampling rates, as demonstrated in §5.2.3.

4.5 Rolling Hash Metadata Management

To eliminate the rolling hash computation of base chunks during delta encoding, FastDelta persists these hashes, as their reuse may occur after unpredictable long intervals. However, this design introduces both storage overhead for maintaining the hashes and additional I/O costs for retrieval.

Storage overhead. Even with sparse hash sampling, each unique chunk (candidate of base chunks) must still retain hash metadata of size $S = \texttt{chunk\_size} \times \texttt{sampling\_rate} \times (8B_{\text{hash}} + 4B_{\text{pos}})$. For an average chunk size of 8 KB and a sampling rate of 1/256, S is approximately 384 B: about 19× larger than secure fingerprints (e.g., 20 B for SHA-1). This significantly inflates metadata storage and undermines compression efficiency. Fortunately, prior studies [15, 52] show that most base chunks originate from the immediately preceding backup version. Leveraging this property, FastDelta applies lifecycle-based hash retention, where rolling hashes are retained only for the latest version's base chunks. In systems like MeGA [52], data migration between versions ensures a restore-friendly layout; FastDelta integrates with this migration process to discard outdated hash metadata, bounding the extra storage overhead to a constant level (§5.2.4). As a result, when writing containers to disk, FastDelta incurs only an additional $\frac{384B}{8KB} \approx 5\%$ of data, leading to negligible write I/O overhead.

I/O overhead. To reduce retrieval costs, FastDelta colocates rolling hashes with their corresponding chunks inside the same container. During delta encoding, these hashes are fetched together with base chunks via piggybacking I/O, eliminating additional disk seeks. As noted earlier, the 5% increase in data volume per read has little measurable impact on container read latency, as confirmed in §5.2.4.

5 Performance Evaluation

5.1 Evaluation Setup

Evaluations are run on a workstation with Ubuntu 18.04, an Intel Xeon(R) Silver 4215R CPU, 32GB memory, and a 16TB 7200RPM HDD, compiled with GCC -O3. In the evaluation, five approaches are considered and implemented according to related papers:

- **CLD**: Chunk-level deduplication, which is configured as an SOTA open source framework, called MFdedup [53], is considered as a baseline of backup throughput.

Name	Original Size	Versions	Workload Descriptions
CHM	279 GB	100	Backups of Source code of Chromium. project from v82.0.4066 to v85.0.4165
WEB	278 GB	102	Backups of website: news.sina.com, captured from Jun. to Sep. in 2016.
LNX	116 GB	266	Backups of Linux kernel source code. Each version is packaged as a tar file.
SYN	431 GB	90	90 synthetic backups by simulating file create/delete/modify operations [38].

Table 2. Four datasets used in the evaluation. *Three are real-world datasets from website snapshots and public projects. One is synthesized on a study of realistic backup storage [38].*

- **MeGA**: A recent high-performance delta compression framework [52] and one of the baselines. It deduplicates chunks and detects similarity between adjacent backups.
- **LoopDelta**: A SOTA delta compression framework [48], which piggybacks the base chunk into fingerprint I/O [1].
- **M_FastDelta/L_FastDelta**: We implement our design of FastDelta based on MeGA and LoopDelta, respectively.

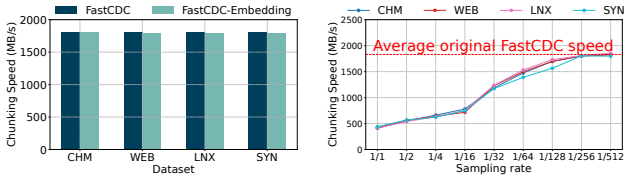
As recommended in existing works [48, 52], they all follow these default configurations: ❶ FastCDC [46] is used for chunking with the minimum, average, and maximum chunk sizes of 1KB, 8KB, and 64KB. SHA1 [13] is used for chunk secure fingerprint. ❷ MeGA and LoopDelta use Odess [51] to generate 12 similar features, which are grouped as 3 "super features" to expedite feature comparison, Xdelta [24] as the delta encoder. Base chunks are only selected from unique chunks. ❸ Consecutive chunks are compressed together with general compressor ZSTD [9] and stored in containers. ❹ The default hash sampling rate of FastDelta is 1/256, as it achieves a balanced performance, as studied in §5.3.

To focus on the performance of the deduplication storage side (i.e., HDD, typical backup media), datasets are sequentially backed up from simulated User Space (a RamDisk) to Backup Space (a 7200rpm HDD), as measured in [52, 53]. Before each backup, the file system cache is flushed by the command "`echo 3 > /proc/sys/vm/drop_caches`", to better reflect the I/O cost. The evaluated datasets, listed in Table 2, are widely used in backup studies [46, 48, 52, 53].

5.2 Micro-style Performance Study of FastDelta

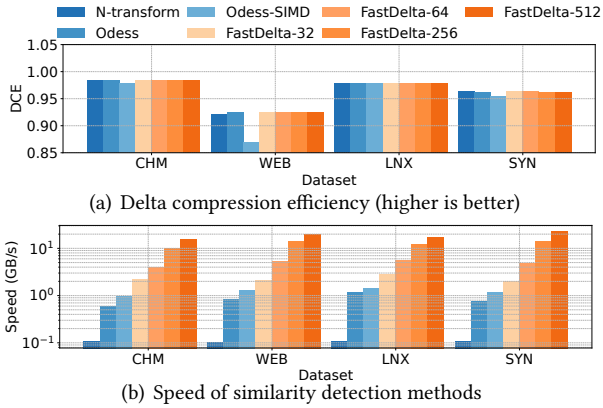
5.2.1 CDC Speed. This subsection investigates the efficiency of embedding hash sampling judgment into chunking judgment in CDC. As shown in Figure 9(a), at the default sampling rate of 1/256, our FastCDC-embedding approach achieves chunking performance close to the original FastCDC. Extra hash sampling operations in CDC introduce only negligible overhead across four datasets. This is due to two factors: (1) the small sampling rate effectively reduces

[1]We implement LoopDelta within the MeGA framework since it is not open-source. To concentrate the discussion of backup throughput, we exclude compressibility-related features (e.g., reverse delta compression and dual similarity track) and keep the piggybacks I/O of base chunks optimization.



(a) FastCDC vs embedding sampling (b) Different sampling rates

Figure 9. Chunking speed of original FastCDC and our embedded sampling variant FastCDC in FastDelta at different sampling rates.



(a) Delta compression efficiency (higher is better)

(b) Speed of similarity detection methods

Figure 10. Performance of FastDelta and SOTA similarity detection methods. FastDelta-N: sampling rate is 1/N.

frequent memory accesses for recording hash, and (2) the embedded judgment ensures that most bytes during chunking undergo only one sampling judgment, triggering chunking decisions only when the sampling condition is satisfied. Furthermore, our embedded hash judgment technique does not alter the original chunking conditions, ensuring consistent chunk boundaries in FastCDC, thereby achieving the same chunk-level deduplication ratio, as evaluated in §5.3.

Figure 9(b) shows the speed of FastCDC-embedding at different sampling rates. The CDC speed degrades as the sampling rate increases. Typically, at a sampling rate of 1, the method degrades to the dual judgment and full hash recording approach, resulting in an 83% decrease in CDC speed. This degradation will severely hinder our effort to accelerate delta compression, which makes CDC become the new computation bottleneck, since CDC processes entire backup streams whereas delta compression only processes non-identical chunks. However, FastDelta leverages embedded hash sampling operations and sparse sampling rates to achieve comparable CDC speed with minimal memory overhead (see §3.3), enabling efficient rolling hash reuse.

5.2.2 Similarity Detection Efficiency. Figure 10(a) shows the delta compression efficiency (DCE, see §3.3) of FastDelta and three other SOTA chunk-level similarity detection methods (N-transform [8], Odess [51], and Odess-SIMD [45], which uses SIMD to accelerate Gear rolling hash), with delta encoder Xdelta [24]. This suggests that FastDelta achieves comparable high DCE compared to other SOTA methods at different sampling rates. The reason is that extracting a maximum value as the similarity feature, either from all rolling hashes or from the content-based sampled hashes, results in comparable similarity detection capabilities, as studied in [51].

In terms of speed, Figure 10(b) suggests that FastDelta is an order of magnitude (10×) faster than Odess and two orders of magnitude (100×) faster than N-transform, achieving ~10 GB/s at 1/256 sampling rate. High-performance comes from two aspects: first, FastDelta completely eliminates the computation of rolling hashes, which accounts for 90% of the computation in Odess, configured with a lightweight rolling hash, Gear [43]. Second, hash sampling results in significantly fewer linear transformations compared to N-transform, which performs linear transformations on all rolling hashes. In contrast, Odess-SIMD simultaneously computes 4 overlapping sliding-window hashes with SIMD to simulate byte-wise single window results , but yields minor speedups (1.5×) than FastDelta. Additionally, it adapts the Gear function to SIMD, which slightly degrades DCE. In summary, FastDelta significantly accelerates similarity detection, without compromising the ability to detect similar chunks.

5.2.3 Delta Encoding Efficiency. This subsection investigates the efficiency of delta encoding and its compressibility when combined with container compression in FastDelta.

Figure 11(a) provides the DCE results of three delta encoding methods, i.e., FastDelta, Xdelta [24], and Gdelta [36]. Only "Xdelta w/ inner" performs inner-chunk compression, while the other methods only perform inter-chunk compression between similar chunks during the delta encoding stage. This suggests that the DCE of FastDelta decreases as the sampling rate increases, with a slower rate of decline. For example, at a small sampling rate such as 1/256, the DCE only decreases by 4%–9% compared to Xdelta (full matching, w/o inner) and Gdelta (with a step-based sampling rate of 2). In contrast, Gdelta's step-based sampling word matching leads to a DCE drop by 42% (as studied in §3.3). This indicates that content-based sampling word-matching achieves more stable compressibility, even at small sampling rates.

To mitigate the compressibility gap introduced by FastDelta's hash sampling, we target compression opportunities at inner-chunk redundancy. For example, Xdelta introduces inner-chunk compression and yields a 3%–26% improvement in DCE. Therefore, we propose disabling inner-chunk compression and entropy encoding in the traditional delta encoding phase, enabling the container compression phase to better exploit inner-chunk redundancy between adjacent chunks via locality-aware container compression. As shown in Figure 11(c), "Xdelta w/o inner" could achieve a higher

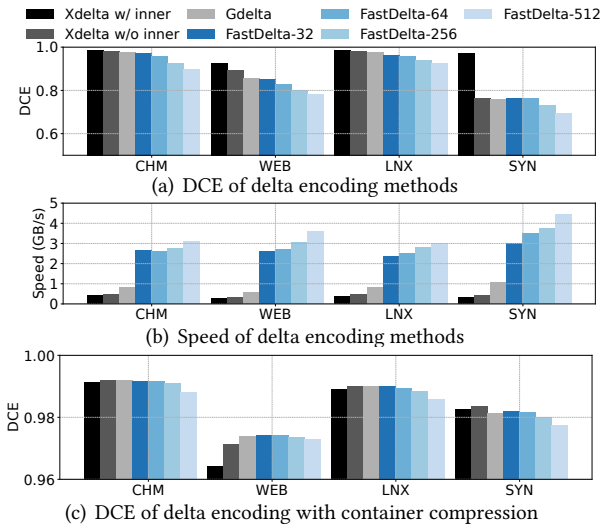


Figure 11. Performance of different delta encoding methods.

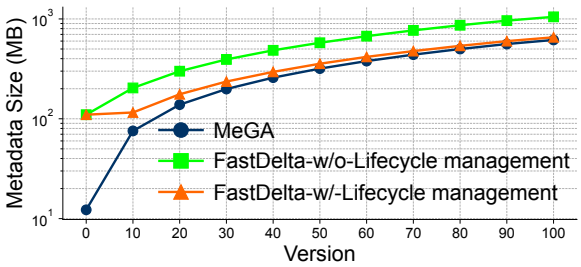


Figure 12. Data size of metadata (fingerprint, sketch, and rolling hash in FastDelta) for increasing backup versions.

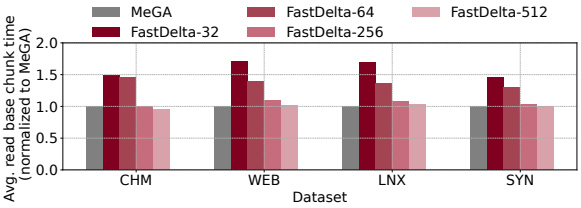


Figure 13. I/O time of reading base chunks .

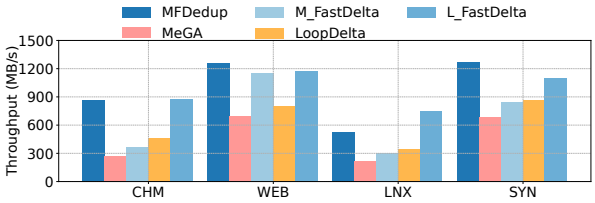


Figure 14. System backup throughput.

DCE compared to "Xdelta w/ inner" combined with container compression. With this technique, FastDelta achieves a comparable compressibility to that of "Xdelta w/ inner". For instance, with a sampling rate of 1/64, FastDelta even surpasses it in DCE across the CHM, WEB, and LNX datasets. With a sampling rate of 1/256, the DCE gap between FastDelta and Xdelta narrows to 0.03%–0.27%.

Figure 11(b) shows that FastDelta achieves substantially higher delta encoding speed compared to Xdelta and Gdelta, with speedups of 5×–9× and 3×–5×, respectively. These gains primarily stem from FastDelta's elimination of rolling hash computations and its use of a small hash sampling rate, which together reduce the overhead of hash indexing in base chunks and hash lookups and content comparisons in input chunks. In summary, by eliminating rolling hash computation, leveraging content-sampled word matching, and locality-aware container compression, FastDelta achieves multi-fold speedup in delta encoding while mitigating DCE degradation due to hash sampling.

5.2.4 Rolling Hash Management Efficiency.

This subsection investigates the efficiency of lifecycle-based hash retention and piggybacking I/O of retrieving rolling hash.

Figure 12 shows the total metadata size on the CHM dataset. This indicates that FastDelta without lifecycle-based rolling hash management generates 82.2% extra metadata overhead at a 1/256 sampling rate, compared with the baseline method (MeGA). This overhead increases further with larger sampling rates. In contrast, FastDelta with lifecycle management can effectively eliminate metadata and produce only 4.7% extra metadata, at a stable, constant level. This successfully solves the metadata explosion problem. Meanwhile, Figure 13 shows that, with piggybacking I/O technique, the extra reading base chunks I/O time introduced by FastDelta-256 is negligible, ranging from 3% to 7%. This is attributed to the elimination of extra disk-seeking time via piggybacking I/O, and the sparse sampling rate, which reduces the volume of rolling hash metadata to be read, approximately 5% of the container, as described in §4.5.

5.3 Comprehensive Evaluation of FastDelta

System backup throughput. Figure 14 suggests that FastDelta achieves faster throughput than other delta compression approaches on four datasets and is close to that of chunk-level deduplication (CLD), with a performance variation ranging from -14%–+42%. Specifically, configured with FastDelta, it improves MeGA's backup throughput by 1.36× (CHM), 1.65× (WEB), 1.40× (LNX), and 1.23× (SYN), respectively. It also accelerates LoopDelta by 1.91× (CHM), 1.46× (WEB), 2.15× (LNX), and 1.26× (SYN), respectively. The improvement comes from the accelerated similarity detection and delta encoding techniques in FastDelta, as evaluated in §5.2. LoopDelta is more efficient than MeGA because it reduces the read I/O of base chunks by piggybacking them on the fingerprint I/O routine. Hence, our computation optimization achieves a larger improvement on it. Furthermore, delta compression reduces write I/O time by achieving a higher deduplication ratio, which makes FastDelta faster than deduplication on LNX and highly competitive on CHM and WEB.

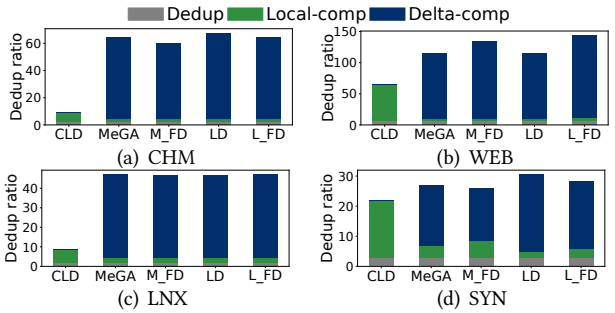


Figure 15. Overall deduplication ratio.

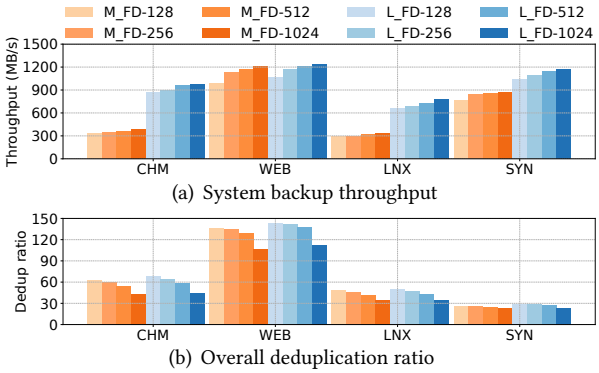


Figure 16. System performance at various sampling rates.

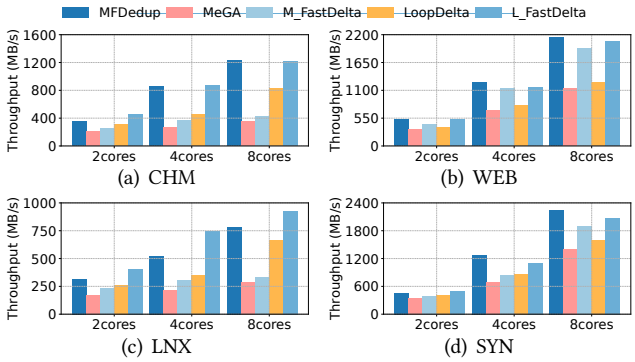


Figure 17. Backup throughput under different CPU core configurations. *The prototype system is configured with 4 cores by default. Typically, it adopts a multi-threaded implementation, dividing the delta compression process into four main task-pipelines: chunking, fingerprinting, deduplicating, and writing. As a result, data chunks can be processed in parallel and flow through different pipelines. For comparison, we restrict the system to 2 cores, and then scale it to 8 cores.*

Overall deduplication ratio. The deduplication ratio is calculated by $\frac{\text{data size before deduplication}}{\text{data size after deduplication}}$, excluding metadata, which is separately evaluated in §5.2.4. Figure 15 suggests that FastDelta maintains the high deduplication ratio of delta compression, which is 1.2×–6.8× higher than that of CLD. Compared to other delta compression approaches, FastDelta exhibits a marginal 1%–7% decrease in CHM, LNX, and SYN, while achieving a 20% higher deduplication ratio in the WEB dataset. This comparable deduplication ability is attributed to its locality-aware container compression, which effectively explores the inner-chunk redundancy and also the content-based sampling word-matching scheme, which maintains stable DCE at sparse sampling rates. In summary, FastDelta mitigates the computation bottleneck of traditional delta compression while preserving its compressibility advantages.

Sensitive study of sampling rates. As discussed in S3.3, to make rolling hash reuse practical, a sparse sampling rate is necessary for reducing the memory consumption (e.g., 1/256 leads to 5% extra memory). This is along with the byproduct advantage of higher speed of similarity detection (§5.2.2) and delta encoding (§5.2.3), less metadata storage, and smaller piggybacking retrieve I/O size (§5.2.4), but it incurs the compression loss of delta encoding (§5.2.3). Thus, we propose content-based sampled word-matching and locality-aware container compression to compensate for this degradation (§5.2.3). Meanwhile, we propose the embedded sampling to reserve high CDC speed (§5.2.1). Figure 16 shows the system performance of FastDelta at various sparse sampling rates. This indicates that FastDelta achieves a stable performance at sparse sampling (except the extremely sparse 1/1024 sampling rate), ranging backup throughput 6%–18% and deduplication ratio 2%–16%. We select 1/256 as the default sampling rate in FastDelta, as it achieves a "sweet spot" between performance gains and data reduction effectiveness across datasets. Adaptively selecting sampling rates may offer further balanced benefits, which we leave as future work.

Backup throughput under different CPU core configurations. Figure 17 shows that: (1) The throughput of all methods increases as more CPU cores are allocated. (2) However, FastDelta directly accelerates delta compression under the same CPU resources. Typically, FastDelta accelerates MeGA by 13%–68% and LoopDelta by 20%–115%. (3) FastDelta with fewer CPU cores achieves comparable performance to SOTA methods with more cores. For instance, MeGA and LoopDelta with 8 cores perform similarly to FastDelta with only 4 cores, in the CHM, WEB, and LNX datasets. In summary, the benefits of FastDelta are consistently observed across different CPU core configurations, as it effectively reduces redundant computations by rolling hash reuse.

6 Discussions

Rolling hash retention and migration. FastDelta extends the existing offline container migration mechanism [52] for lifecycle-based rolling hash retention. Typically, when containers are migrated, the latest rolling hashes are moved together with the corresponding chunk at slight I/O cost. For non-migrated containers, FastDelta stores rolling hashes in the container with the format [ChunkData | HashData | HashSize], allowing old hashes to be removed via a simple

truncate operation without requiring reads/rewrites. In our testing, FastDelta incurs 6%–10% extra migration overhead.

Restore overhead. Based on the rolling hash retention, FastDelta adds a small fraction of rolling hash metadata (only ~5%, see §4.5) to the deduplicated data of the latest backup version, while earlier versions remain unchanged. Besides, FastDelta does not change the original restore workflow. As a result, restoring only the latest backup requires minimal extra reads in FastDelta, and its restore performance is comparable to the advanced prototype system (i.e., MeGA).

7 Conclusion

This paper presents FastDelta, a novel delta compression approach that reuses rolling hash results to eliminate redundant computations across multiple stages. It incorporates three key techniques: embedded hash sampling, locality-aware container compression, and lifecycle-based hash retention with piggybacked retrieval for long-term reuse. Experimental results show that FastDelta delivers throughput comparable to chunk-level deduplication while maintaining the high deduplication ratios of delta compression.

Acknowledgment

We sincerely thank our shepherd, Jacob Gorm Hansen, and the anonymous reviewers for helping us improve our paper significantly. This research was partly supported by the Major Key Project of PCL under Grant PCL2024A05, the National Natural Science Foundation of China under Grants 62472127 and 62502119, GuangDong Basic and Applied Basic Research Foundation under Grant 2023A1515110072, and Shenzhen Science and Technology Program under Grant KJZD20231023094701003.

References

[1] 2016. Deduplication document of Ceph. *https://docs.ceph.com/en/latest/dev/deduplication* (2016).

[2] 2025. Google file transfer. *https://github.com/google/cdc-file-transfer* (2025).

[3] 2025. ZSTD. *https://github.com/facebook/zstd* (2025).

[4] Inc. Advanced Micro Devices. 2007. *AMD64 Technology: textttCMOVcc — Conditional Move.* Technical Report 24592 Rev. 3.14. AMD. https://www.cs.tufts.edu/comp/40-2011f/readings/amd-cmovcc.pdf

[5] Yamini Allu, Fred Douglis, Mahesh Kamat, Philip Shilane, Hugo Patterson, and Ben Zhu. 2017. Backup to the future: How workload and hardware changes continually redefine data domain file systems. *Computer* 50, 7 (2017), 64–72.

[6] George Amvrosiadis and Medha Bhadkamkar. 2015. Identifying trends in enterprise data protection systems. In *Proceedings of the 2015 USENIX Annual Technical Conference (USENIX ATC' 15)*. 151–164.

[7] Andrei Z Broder. 1997. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences (SEQUENCES' 97)*. IEEE, 21–29.

[8] Andrei Z Broder. 2000. Identifying and filtering near-duplicate documents. In *Annual symposium on combinatorial pattern matching*. Springer, 1–10.

[9] Y Collet and M Kucherawy. 2018. Zstandard Compression and the application/zstd Media Type. *RFC 8478* (2018).

[10] Dell EMC. 2025. Data Domain - Data Backup Appliance, Data Protection. https://www.dellemc.com/en-us/data-protection/data-domain-backup-storage.htm

[11] Cai Deng, Xiangyu Zou, Qi Chen, Bo Tang, and Wen Xia. 2024. The design of a lossless deduplication scheme to eliminate fine-grained redundancy for JPEG image storage systems. *IEEE Trans. Comput.* 73, 5 (2024), 1385–1399.

[12] Abhinav Duggal, Fani Jenkins, Philip Shilane, Ramprasad Chinthekindi, Ritesh Shah, and Mahesh Kamat. 2019. Data domain cloud tier: Backup here, backup there, deduplicated everywhere!. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC' 19)*. 647–660.

[13] D Eastlake 3rd and Paul Jones. 2001. Rfc3174: Us secure hash algorithm 1 (sha1).

[14] Min Fu, Dan Feng, Yu Hua, Xubin He, Zuoning Chen, Wen Xia, Yucheng Zhang, and Yujuan Tan. 2015. Design tradeoffs for data deduplication performance in backup workloads. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST' 15)*. 331–344.

[15] Hongming Huang, Peng Wang, Qiang Su, Hong Xu, Chun Jason Xue, and André Brinkmann. 2024. Palantir: Hierarchical Similarity Detection for Post-Deduplication Delta Compression. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 830–845.

[16] David A Huffman. 1952. A method for the construction of minimum redundancy codes. *Institute of Radio Engineers (IRE)* 40, 9 (1952), 1098–1101.

[17] Jürgen Kaiser, André Brinkmann, Tim Süß, and Dirk Meister. 2015. Deriving and comparing deduplication techniques using a model-based classification. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*. 1–13.

[18] Richard M Karp and Michael O Rabin. 1987. Efficient randomized pattern-matching algorithms. *IBM journal of research and development* 31, 2 (1987), 249–260.

[19] Iwona Kotlarska, Andrzej Jackowski, Krzysztof Lichota, Michal Welnicki, Cezary Dubnicki, and Konrad Iwanicki. 2023. InftyDedup: Scalable and Cost-Effective Cloud Tiering with Deduplication. In *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST' 23)*. 33–48.

[20] Jingwei Li, Zuoru Yang, Yanjing Ren, Patrick PC Lee, and Xiaosong Zhang. 2020. Balancing storage efficiency and data confidentiality with tunable encrypted deduplication. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*. 1–15.

[21] Pengfei Li, Yu Hua, Qin Cao, and Mingxuan Zhang. 2020. Improving the restore performance via physical-locality middleware for backup systems. In *Proceedings of the 21st International Middleware Conference (Middleware' 20)*. 341–355.

[22] Mark Lillibridge, Kave Eshghi, and Deepavali Bhagwat. 2013. Improving restore speed for backup systems that use inline chunk-based deduplication. In *Proceedings of the 11th USENIX conference on file and storage technologies (FAST' 13)*.

[23] Dingbang Liu, Xiangyu Zou, Tao Lu, Philip Shilane, Wen Xia, Wenxuan Huang, Yanqi Pan, and Hao Huang. 2025. Garbage Collection Does Not Only Collect Garbage: Piggybacking-Style Defragmentation for Deduplicated Backup Storage. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. 1026–1043.

[24] J. MacDonald. 2000. File system support for delta compression. Masters thesis. Department of Electrical Engineering and Computer Science, University of California at Berkeley.

[25] Konstantinos Margaritis. 2005. Vectorization of Algorithm Adler32 Using Altivec. *Nafplion, Greece, May* 15 (2005).

[26] Jaehong Min, Daeyoung Yoon, and Youjip Won. 2011. Efficient deduplication techniques for modern backup operation. *IEEE Trans. Comput.*

60, 6 (2011), 824–840.

[27] Athicha Muthitacharoen, Benjie Chen, and David Mazières. 2001. A Low-Bandwidth Network File System. In *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles (SOSP' 01)*. 174–187.

[28] NetApp Inc. 2018. ONTAP Data Management Software: ONTAP Data Management Software. https://www.netapp.com/us/products/data-management-software/ontap.aspx Accessed: 2024-12-26.

[29] Fan Ni and Song Jiang. 2019. RapidCDC: Leveraging duplicate locality to accelerate chunking in CDC-based deduplication systems. In *Proceedings of the 2019 ACM symposium on cloud computing (SoCC' 19)*. 220–232.

[30] Myoungwon Oh, Sungmin Lee, Samuel Just, Young Jin Yu, Duck-Ho Bae, Sage Weil, Sangyeun Cho, and Heon Y Yeom. 2023. TiDedup: A New Distributed Deduplication Architecture for Ceph. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC' 23)*. 117–131.

[31] Michael O Rabin. 1981. *Fingerprinting by random polynomials*. Center for Research in Computing Techn., Aiken Computation Laboratory, Univ.

[32] David Reinsel, John Gantz, and John Rydning. 2018. The digitization of the world from edge to core. *IDC White Paper* (2018).

[33] Phlip Shilane, Mark Huang, Grant Wallace, and Windsor Hsu. 2012. WAN-optimized replication of backup datasets using stream-informed delta compression. *ACM Transactions on Storage (TOS)* 8, 4, Article 13 (November 2012), 26 pages.

[34] Philip Shilane, Grant Wallace, Mark Huang, and Windsor Hsu. 2012. Delta compressed and deduplicated storage using stream-informed locality. In *Proceedings of the 2012 USENIX Conference on Hot Topics in Storage and File Systems (HotStorage'12)*.

[35] Tong Sun, Bowen Jiang, Borui Li, Jiamei Lv, Yi Gao, and Wei Dong. 2024. {SimEnc}: A {High-Performance}{Similarity-Preserving} Encryption Approach for Deduplication of Encrypted Docker Images. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC' 24)*. 615–630.

[36] Haoliang Tan, Wen Xia, Xiangyu Zou, Cai Deng, Qing Liao, and Zhaoquan Gu. 2024. The Design of Fast Delta Encoding for Delta Compression Based Storage Systems. *ACM Transactions on Storage (TOS)* 20, 4 (2024), 1–30.

[37] Haoliang Tan, Xiangyu Zou, Binzhaoshuo Wan, Zhaoquan Gu, and Wen Xia. 2024. SuperDelta: Multiple Referenced Base Chunks Scheme for Fine-grained Deduplication Backup Storage System. In *Proceedings of the 2024 Data Compression Conference (DCC' 24)*. IEEE, 362–371.

[38] Vasily Tarasov, Amar Mudrankit, Will Buik, Philip Shilane, Geoff Kuenning, and Erez Zadok. 2012. Generating realistic datasets for deduplication analysis. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC' 12)*. 261–272.

[39] Grant Wallace, Fred Douglis, Hangwei Qian, Philip Shilane, and Windsor Hsu. 2012. Characteristics of Backup Workloads in Production Systems. In *Proceedings of the 10th USENIX conference on File and Storage Technologies (FAST' 12)*.

[40] Mingze Xia, Sheng Di, Franck Cappello, Pu Jiao, Kai Zhao, Jinyang Liu, Xuan Wu, Xin Liang, and Hanqi Guo. 2024. Preserving topological feature with sign-of-determinant predicates in lossy compression: A case study of vector field critical points. In *Proceedings of 40th International Conference on Data Engineering (ICDE' 24)*. IEEE, 4979–4992.

[41] Mingze Xia, Bei Wang, Yuxiao Li, Pu Jiao, Xin Liang, and Hanqi Guo. 2025. Tspsz: An efficient parallel error-bounded lossy compressor for topological skeleton preservation. In *Proceedings of 41th International Conference on Data Engineering (ICDE' 25)*. IEEE, 3682–3695.

[42] Wen Xia, Hong Jiang, Dan Feng, and Lei Tian. 2015. DARE: A deduplication-aware resemblance detection and elimination scheme for data reduction with low overheads. *IEEE Trans. Comput.* 65, 6 (2015), 1692–1705.

[43] Wen Xia, Hong Jiang, Dan Feng, Lei Tian, Min Fu, and Yukun Zhou. 2014. Ddelta: A deduplication-inspired fast delta compression approach. *Performance Evaluation* 79 (2014), 258–272.

[44] Wen Xia, Chunguang Li, Hong Jiang, Dan Feng, Yu Hua, Leihua Qin, and Yucheng Zhang. 2015. Edelta: A Word-enlarging Based Fast Delta Compression Approach. In *Proceedings of the the 7th USENIX conference on Hot Topics in Storage and File Systems (HotStorage' 15)*. 1–5.

[45] Wen Xia, Lifeng Pu, Xiangyu Zou, Philip Shilane, Shiyi Li, Haijun Zhang, and Xuan Wang. 2023. The design of fast and lightweight resemblance detection for efficient post-deduplication delta compression. *ACM Transactions on Storage (TOS)* 19, 3 (2023), 1–30.

[46] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. 2016. FastCDC: A Fast and Efficient Content-Defined Chunking Approach for Data Deduplication. In *Proceedings of the 2016 USENIX Annual Technical Conference (USENIX ATC' 16)*. 101–114.

[47] Jingyuan Yang, Jun Wu, Ruilin Wu, Jingwei Li, Patrick PC Lee, Xiong Li, and Xiaosong Zhang. 2025. {ShieldReduce}:{Fine-Grained} Shielded Data Reduction. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC' 25)*. 1281–1296.

[48] Yucheng Zhang, Hong Jiang, Dan Feng, Nan Jiang, Taorong Qiu, and Wei Huang. 2023. LoopDelta: Embedding Locality-aware Opportunistic Delta Compression in Inline Deduplication for Highly Efficient Data Reduction. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC' 23)*. 133–148.

[49] Yucheng Zhang, Wen Xia, Dan Feng, Hong Jiang, Yu Hua, and Qiang Wang. 2019. Finesse: fine-grained feature locality based fast resemblance detection for post-deduplication delta compression. In *Proceedings of USENIX Conference on File and Storage Technologies (FAST'19)*. 121–128.

[50] Benjamin Zhu, Kai Li, and R Hugo Patterson. 2008. Avoiding the disk bottleneck in the data domain deduplication file system.. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST'08)*. 1–14.

[51] Xiangyu Zou, Cai Deng, Wen Xia, Philip Shilane, Haoliang Tan, Haijun Zhang, and Xuan Wang. 2021. Odess: Speeding up resemblance detection for redundancy elimination by fast content-defined sampling. In *Proceedings of the 2021 IEEE 37th International Conference on Data Engineering (ICDE' 21)*. IEEE, 480–491.

[52] Xiangyu Zou, Wen Xia, Philip Shilane, Haijun Zhang, and Xuan Wang. 2022. Building a high-performance fine-grained deduplication framework for backup storage with high deduplication ratio. In *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC' 22)*. 19–36.

[53] Xiangyu Zou, Jingsong Yuan, Philip Shilane, Wen Xia, Haijun Zhang, and Xuan Wang. 2021. The dilemma between deduplication and locality: Can both be achieved?. In *Proceedings of 19th USENIX conference on file and storage technologies (FAST' 21)*. 171–185.